

JHS

JHS Web

Cofradía Jesús Salvador de los Hombres

Memoria técnica adaptada para Trabajo Fin de Grado

Titulación	Trabajo Fin de Grado / Proyecto académico
Autor/a	[Nombre y apellidos]
Tutor/a	[Nombre del tutor/a]
Centro	[Universidad / Centro]
Lugar y fecha	Montijo, Badajoz · Mayo 2026

Resumen

Este documento presenta la memoria técnica de JHS Web, una plataforma digital desarrollada para la Cofradía Jesús Salvador de los Hombres de Montijo. La aplicación reúne en un mismo entorno la gestión de hermanos, la publicación de noticias, una tienda online, el pago de la cuota anual y el seguimiento GPS de la procesión en tiempo real.

La memoria recoge el análisis de requisitos, los principales casos de uso, el modelo de datos, la arquitectura del sistema y las pruebas realizadas. Además, se ha reorganizado el contenido para que la lectura sea más clara y se ha incorporado un formato académico, manteniendo una identidad visual coherente con el proyecto: verde oscuro, dorado y fondos claros.

PALABRAS CLAVE

JHS Web; cofradía; gestión de usuarios; React; Supabase; Stripe; geolocalización; accesibilidad; aplicación web.

Abstract

This document presents the technical report of JHS Web, a digital platform developed for the Cofradía Jesús Salvador de los Hombres in Montijo. The application integrates member management, news publication, an online store, annual fee payment and real-time GPS tracking of the procession.

The report describes the system requirements, main use cases, data model, architecture and validation tests. The content has been reorganised into an academic structure while preserving the visual identity of the project through a sober palette based on dark green, gold and light neutral backgrounds.

KEYWORDS

JHS Web; brotherhood; user management; React; Supabase; Stripe; geolocation; accessibility; web application.

Índice de contenidos

1. Introducción
2. Justificación y objetivos
3. Descripción general del sistema
4. Requisitos del sistema
5. Análisis de casos de uso
6. Diseño de datos
7. Arquitectura del sistema
8. Instalación, ejecución y despliegue
9. Validación y pruebas realizadas
10. Seguridad, accesibilidad y protección de datos
11. Limitaciones y líneas de mejora
12. Conclusiones
13. Bibliografía

Nota de formato

El índice se ha dejado en una estructura limpia y editable. Si se abre el archivo DOCX en Word, puede sustituirse por una tabla de contenido automática usando los estilos de título ya configurados.

1. Introducción

JHS Web es una plataforma digital creada para mejorar la organización interna y la comunicación de la Cofradía Jesús Salvador de los Hombres de Montijo (Badajoz). El sistema permite centralizar tareas que tradicionalmente se gestionan de forma separada, como el padrón de hermanos, el pago de cuotas, la publicación de noticias, la tienda de la cofradía y el seguimiento de la procesión.

El desarrollo se ha planteado como una aplicación web moderna, accesible desde distintos dispositivos y preparada para responder a un contexto real de uso. La propuesta no se limita a presentar una página informativa, sino que incorpora funcionalidades privadas para hermanos y herramientas específicas para la administración de la cofradía.

Desde el punto de vista técnico, el proyecto combina tecnologías de frontend actuales con servicios backend en la nube. Esta elección busca facilitar el mantenimiento, reducir la complejidad de infraestructura y permitir que la aplicación pueda escalar de forma progresiva si aumentan los usuarios o las necesidades de la entidad.

2. Justificación y objetivos

La digitalización de una cofradía no consiste únicamente en trasladar trámites a internet. También supone hacer más sencilla la participación de los hermanos, mejorar la trazabilidad de la información y ofrecer una comunicación más clara con la comunidad. En este sentido, JHS Web nace como una herramienta práctica para cubrir necesidades concretas y reales.

OBJETIVO GENERAL

Diseñar y desarrollar una aplicación web que permita gestionar de manera centralizada la actividad digital de la cofradía, integrando funcionalidades públicas, privadas y administrativas en un entorno seguro y fácil de utilizar.

OBJETIVOS ESPECÍFICOS

- Digitalizar el registro y seguimiento del padrón de hermanos.
- Permitir el pago online de la cuota anual y de productos de la tienda.
- Ofrecer noticias y documentos de interés para visitantes y hermanos.
- Facilitar el seguimiento GPS de la procesión en tiempo real.
- Incorporar un panel de administración para la gestión diaria de la plataforma.
- Garantizar una experiencia responsive, accesible y coherente con la identidad visual de la cofradía.

3. Descripción general del sistema

La aplicación se organiza en dos grandes áreas. La primera corresponde a la parte pública, accesible para cualquier visitante, donde se pueden consultar contenidos informativos y visualizar la procesión. La segunda corresponde al área privada, destinada a hermanos registrados y administradores, con funcionalidades condicionadas por el rol y el estado de cada usuario.

3.1. Actores del sistema

Actor	Descripción
Visitante	Usuario sin cuenta. Puede acceder a la información pública, noticias y seguimiento de la procesión.
Hermano	Usuario registrado con acceso al área privada. Puede consultar su información y realizar gestiones básicas.
Cofrade activo	Hermano con cuota pagada y estado activo. Puede acceder a funciones específicas de participación en la procesión.
Administrador	Usuario con permisos para gestionar noticias, hermanos, productos, pedidos y seguimiento GPS.
Superadministrador	Perfil con todos los permisos del administrador y capacidad para modificar roles.

3.2. Módulos principales

Módulo	Función dentro del sistema
Gestión de usuarios	Registro, autenticación, estados de hermano, roles y validación de datos personales.
Noticias	Creación, edición, publicación y consulta de noticias con imágenes o documentos adjuntos.
Tienda online	Catálogo de productos, carrito persistente, pedidos e integración con Stripe.
Cuotas	Pago anual online o registro de pago presencial por parte de administración.
GPS procesional	Visualización pública del recorrido y actualización de posición de los pasos en tiempo real.
Panel de administración	Métricas, gestión de hermanos, inventario, pedidos, cuentas y control de permisos.

4. Requisitos del sistema

Los requisitos se han agrupado por áreas funcionales para que el documento resulte más claro. Esta organización permite relacionar cada necesidad del sistema con el módulo que la implementa y facilita su revisión durante la evaluación del proyecto.

4.1. Requisitos funcionales

Código	Área	Descripción
RF-01	Gestión de usuarios	Registro de hermanos con datos personales, subida opcional del certificado de bautismo, inicio de sesión, gestión de roles y estados de hermano.
RF-02	Autenticación y acceso	Protección de rutas privadas, rutas de administración limitadas a roles autorizados y mantenimiento de sesión entre recargas.
RF-03	Noticias	Consulta pública de noticias y administración completa de publicaciones, incluyendo destacados y archivos adjuntos.
RF-04	Tienda online	Catálogo de productos, carrito persistente, pago mediante Stripe, historial de pedidos e inventario administrable.
RF-05	Cuotas	Pago de la cuota anual online o presencial y activación automática del hermano tras el pago.
RF-06	GPS y procesión	Mapa público con recorrido, posición de pasos en tiempo real y control GPS desde dispositivos móviles de administración.
RF-07	Administración	Dashboard con métricas, gestión de puestos de procesión, módulo de cuentas y control de permisos.
RF-08	Accesibilidad	Modo Lectura con aumento de fuente y alto contraste, navegación por teclado y compatibilidad con lectores de pantalla.

4.2. Requisitos no funcionales

Categoría	Criterio aplicado
Rendimiento	Carga inicial inferior a tres segundos en conexión 4G, actualización rápida del GPS y optimización del bundle mediante Vite.
Seguridad	Autenticación gestionada por Supabase Auth, Row Level Security en las tablas, pagos externos con Stripe y protección de variables sensibles.

Disponibilidad	Despliegue en Vercel con CDN global y uso de servicios gestionados para mejorar la continuidad del servicio.
Escalabilidad	Arquitectura serverless y patrón Repository para facilitar futuras sustituciones o ampliaciones del backend.
Mantenibilidad	Código en TypeScript, separación de responsabilidades y organización clara de carpetas y componentes.
Usabilidad	Diseño responsive mobile-first, navegación consistente y mensajes de confirmación mediante toasts.
Compatibilidad	Funcionamiento previsto en navegadores modernos con JavaScript habilitado.
Protección de datos	Páginas legales, tratamiento conforme al RGPD y almacenamiento en región europea cuando el proveedor lo permite.

5. Análisis de casos de uso

Los casos de uso muestran la interacción entre cada actor y el sistema. En lugar de presentarlos como una lista extensa de pasos aislados, se han resumido en una tabla para destacar el objetivo, las condiciones de partida y el resultado esperado de cada proceso.

Código	Caso de uso	Actor principal	Resultado esperado
CU-01	Registrarse en la plataforma	Visitante	Crear cuenta de hermano con estado pendiente de pago.
CU-02	Iniciar sesión	Hermano / Admin	Validar credenciales y redirigir según el rol.
CU-03	Pagar cuota anual	Hermano	Completar pago con Stripe o registro presencial y activar el estado de cofrade.
CU-04	Comprar en la tienda	Cofrade activo	Añadir productos al carrito, pagar y consultar el historial de pedidos.
CU-05	Seguir la procesión	Visitante / Hermano	Consultar en el mapa el recorrido y la posición de los pasos.
CU-06	Controlar GPS de un paso	Administrador	Activar o desactivar el envío de coordenadas desde un móvil.
CU-07	Publicar una noticia	Administrador	Crear una noticia con imagen o PDF y publicarla.
CU-08	Gestionar hermanos	Administrador	Buscar, filtrar, editar datos y revisar estados o roles.
CU-09	Gestionar productos y pedidos	Administrador	Crear productos, actualizar stock y avanzar estados de pedidos.

Lectura del análisis funcional

Cada caso de uso se ha mantenido fiel al comportamiento del sistema, pero se ha redactado con un lenguaje más académico y uniforme. Esta presentación facilita que el evaluador entienda rápidamente qué hace la plataforma y qué valor aporta cada módulo.

6. Diseño de datos

El modelo de datos se apoya en PostgreSQL a través de Supabase. Las tablas principales cubren la gestión de usuarios, noticias, productos, pedidos, recorrido procesional y seguimiento GPS. Además, Supabase Storage se utiliza para almacenar archivos vinculados al registro de hermanos, noticias y productos.

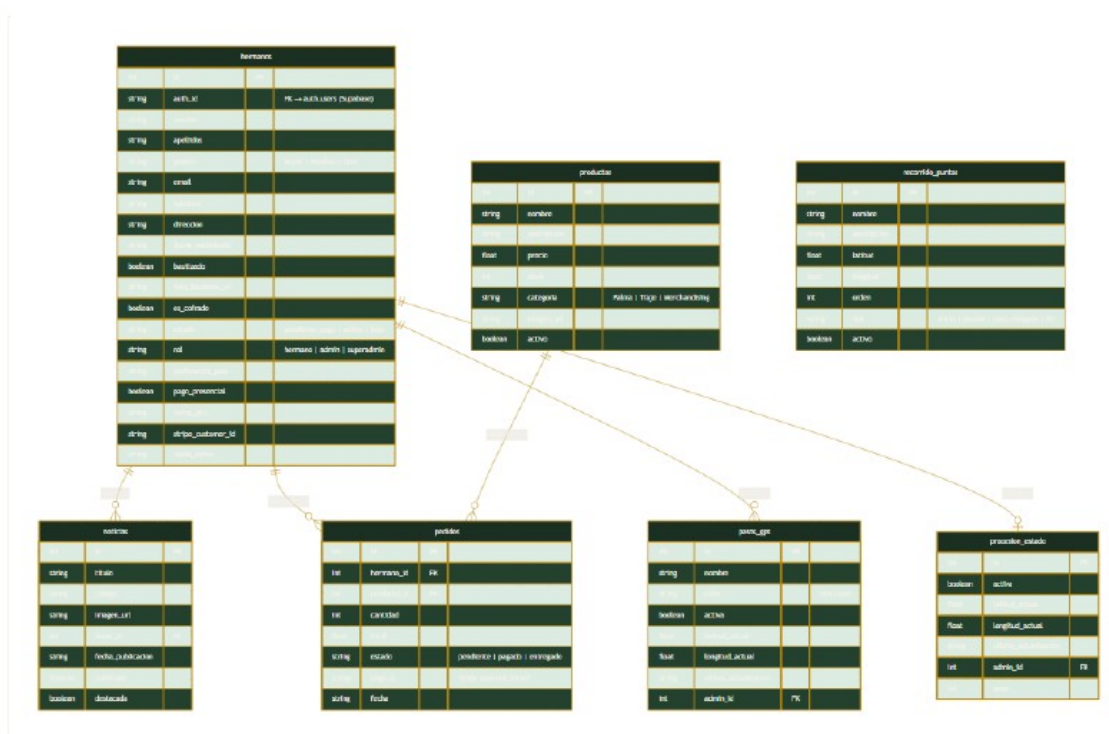


Figura 1. Modelo relacional de la base de datos del sistema JHS Web.

Tabla	Función
hermanos	Usuarios del sistema, datos personales, roles, estado de cofrade y relación con pagos.
noticias	Publicaciones informativas, con un hermano administrador como autor.
productos	Catálogo de tienda con nombre, precio, categoría, stock, imagen y estado activo.
pedidos	Relación entre hermanos y productos, con cantidad,

	estado de pago, fecha y referencia de Stripe.
pasos_gps	Información de los pasos procesionales y coordenadas GPS actualizadas en tiempo real.
procesion_estado	Fila de control del estado general de la procesión y posición de la cruz guía.
recorrido_puntos	Puntos ordenados del recorrido procesional, sin dependencia directa de otras tablas.

6.1. Almacenamiento de archivos y funciones Edge

Bucket	Contenido almacenado
bautismos	Fotos o justificantes de fe de bautismo subidos durante el registro.
noticias	Imágenes y documentos PDF adjuntos a las publicaciones.
productos	Imágenes del catálogo de la tienda online.

Edge Function	Responsabilidad
create-checkout-session	Crea sesiones de pago de Stripe para cuotas o productos del carrito.
stripe-webhook	Recibe eventos de Stripe y actualiza el estado de pedidos o hermanos.

7. Arquitectura del sistema

La arquitectura sigue una separación clara entre interfaz, acceso a datos y servicios externos. El frontend se ha desarrollado con React, TypeScript y Vite, mientras que Supabase proporciona autenticación, base de datos, almacenamiento y funcionalidades en tiempo real. Stripe se utiliza exclusivamente para el procesamiento de pagos.

7.1. Stack tecnológico

Capa	Tecnologías principales
Frontend	React, TypeScript, Vite, Tailwind CSS, Framer Motion, React Router DOM, Zustand, Leaflet, Recharts, lucide-react, react-hot-toast
Backend como servicio	Supabase: PostgreSQL, Auth, Storage, Realtime y Edge Functions
Pagos	Stripe Checkout y webhooks de confirmación
Despliegue	Vercel para la aplicación web y CDN global

7.2. Patrón Repository

El proyecto aplica el patrón Repository para evitar que los componentes de React dependan directamente de Supabase. Los componentes trabajan contra una interfaz o contrato, y la implementación concreta se encarga de comunicarse con la base de datos. Esta decisión mejora la mantenibilidad y facilita futuras pruebas o cambios de proveedor.

```
Componente React
↓ llama a
Interfaz del repositorio (src/database/repositories/)
↓ implementada por
SupabaseXxxRepository (src/database/supabase/)
↓ utiliza
Supabase Client o Supabase Admin según el contexto
```

7.3. Clientes de Supabase

El sistema diferencia entre un cliente público, sujeto a las reglas de seguridad de la base de datos, y un cliente de administración, reservado para operaciones privilegiadas. La clave de servicio debe mantenerse siempre fuera del navegador y configurarse únicamente en entornos de servidor o funciones seguras.

Cliente	Uso recomendado
supabaseClient	Utiliza la clave pública anon y el JWT del usuario. Se emplea en lecturas públicas y operaciones permitidas por RLS.
supabaseAdmin	Utiliza la service_role key. Se reserva para operaciones privilegiadas ejecutadas desde entorno seguro, nunca desde el frontend.

7.4. Flujo GPS en tiempo real

El seguimiento GPS se basa en la API nativa del navegador. Cuando un administrador activa el seguimiento desde un móvil, el sistema obtiene las coordenadas, actualiza la tabla correspondiente en Supabase y propaga los cambios a los visitantes mediante Realtime. Si el canal en tiempo real falla, el mapa mantiene un polling de respaldo.

```
Administrador en móvil → watchPosition() del navegador
Coordenadas → actualización segura en pasos_gps
Supabase Realtime → evento postgres_changes
Visitantes → actualización de marcadores en Leaflet
```

8. Instalación, ejecución y despliegue

La instalación del proyecto requiere Node.js, npm, Git y cuentas en Supabase, Stripe y Vercel. A continuación se recogen los pasos necesarios para ejecutar la aplicación en local y preparar un despliegue básico de producción.

8.1. Requisitos previos

Herramienta	Uso
Node.js	Versión 18 o superior.
npm	Versión 9 o superior, incluido con Node.js.
Git	Necesario para clonar el repositorio.
Supabase	Proyecto gratuito para base de datos, autenticación, Storage y Realtime.
Stripe	Cuenta en modo test para pagos con tarjeta.
Vercel	Cuenta para el despliegue de la aplicación.

8.2. Ejecución local

```
git clone https://github.com/tu-usuario/jhs-web.git
cd jhs-web
npm install
npm run dev
```

Tras ejecutar el servidor de desarrollo, la aplicación queda disponible de forma local en el puerto indicado por Vite, habitualmente `http://localhost:5173`.

8.3. Variables de entorno

En una entrega técnica es importante diferenciar las claves públicas de las claves privadas. Las variables que empiezan por `VITE_` son accesibles desde el frontend; por tanto, nunca deben contener la `service_role key` de Supabase.

```
VITE_SUPABASE_URL=https://tu-proyecto.supabase.co
VITE_SUPABASE_ANON_KEY=tu_anon_key
VITE_GEMINI_API_KEY=tu_gemini_api_key # opcional

# Solo en entorno seguro / Edge Functions / servidor
SUPABASE_SERVICE_ROLE_KEY=tu_service_role_key
STRIPE_SECRET_KEY=sk_test_xxxxx
STRIPE_WEBHOOK_SECRET=whsec_xxxxx
```

8.4. Compilación y despliegue

```
npm run build    # genera el directorio /dist  
npm run preview  # previsualiza el build en local  
npm run lint     # análisis estático con ESLint
```

Para el despliegue, el repositorio puede conectarse a Vercel, añadir las variables de entorno necesarias y activar el despliegue automático en cada actualización de la rama principal.

9. Validación y pruebas realizadas

La validación del sistema se ha realizado mediante pruebas manuales funcionales. Aunque no se configuró una suite automatizada con herramientas como Vitest o Cypress, los casos revisados permiten comprobar el comportamiento principal de la aplicación en las áreas más relevantes.

Punto de mejora reconocido

La ausencia de tests automatizados debe plantearse como una línea clara de mejora futura. Para una evolución del proyecto, sería recomendable incorporar pruebas unitarias de repositorios, pruebas de integración para pagos y pruebas end-to-end de los flujos críticos.

Código	Área probada	Comprobaciones realizadas	Resultado
PT-01	Registro de nuevo hermano	Registro correcto, email ya existente y validación de campos obligatorios.	Correcto
PT-02	Autenticación y acceso	Login correcto, error de contraseña y protección de rutas por rol o estado.	Correcto
PT-03	Gestión de noticias	Creación de noticia con imagen, PDF adjunto y borradores no visibles al público.	Correcto
PT-04	Tienda y carrito	Añadir producto, persistencia en localStorage, checkout con Stripe y gestión de error.	Correcto
PT-05	GPS en tiempo real	Activación, visibilidad pública, desactivación y polling de respaldo.	Correcto
PT-06	Responsive y accesibilidad	Vista móvil, Modo Lectura y navegación por teclado.	Correcto

10. Seguridad, accesibilidad y protección de datos

La seguridad del sistema se apoya en tres ideas principales: autenticación delegada en Supabase Auth, reglas de Row Level Security en la base de datos y procesamiento de pagos fuera de la aplicación mediante Stripe. De esta forma, la plataforma no almacena contraseñas en texto plano ni gestiona datos bancarios de los usuarios.

Respecto a la accesibilidad, el proyecto incorpora un Modo Lectura que aumenta el tamaño de la fuente y mejora el contraste. Además, la navegación por teclado y el diseño responsive permiten utilizar la plataforma en dispositivos móviles y en contextos de accesibilidad variados.

En materia de protección de datos, la aplicación incluye páginas legales de aviso legal, política de privacidad y política de cookies. La memoria también deja constancia de la necesidad de tratar los datos personales de los hermanos conforme al RGPD y a la normativa española aplicable.

11. Limitaciones y líneas de mejora

Todo proyecto académico tiene un alcance concreto. En este caso, la aplicación cubre los flujos principales y demuestra una arquitectura viable, pero todavía puede crecer en varias direcciones. Estas mejoras no invalidan el trabajo realizado; al contrario, ayudan a mostrar una visión crítica y realista del desarrollo.

Línea de mejora	Descripción
Tests automatizados	Añadir pruebas unitarias, de integración y end-to-end para los flujos de registro, pago y administración.
Auditoría de seguridad	Revisar políticas RLS, permisos de Storage, webhooks y gestión de secretos antes de un uso real en producción.
Panel económico	Ampliar el módulo de cuentas con filtros, exportación y generación de informes.
Notificaciones	Incorporar avisos por correo o mensajería para cuotas, pedidos y noticias importantes.
Mejora documental	Añadir capturas de pantalla finales, manual de usuario y anexos técnicos si la entrega lo requiere.

12. Conclusiones

JHS Web demuestra que una entidad local puede disponer de una plataforma digital completa, útil y adaptada a sus necesidades reales sin perder su identidad. El proyecto combina una parte pública de consulta con un área privada para hermanos y un panel de administración que facilita la gestión diaria.

Desde el punto de vista técnico, la elección de React, TypeScript, Supabase, Stripe y Vercel permite construir una aplicación moderna, mantenible y preparada para evolucionar. La separación mediante el patrón Repository, el uso de autenticación gestionada y la integración de servicios externos aportan una base sólida para futuras ampliaciones.

Como resultado, la aplicación no solo resuelve tareas concretas, sino que propone una forma más ordenada, accesible y transparente de relacionar a la cofradía con sus hermanos y visitantes. La memoria deja constancia del trabajo realizado y de los pasos necesarios para seguir mejorando el sistema en un entorno real.

13. Bibliografía

Allsopp, J. (2000). A Dao of Web Design. A List Apart. <https://alistapart.com/article/dao>

Deno. (2024). Deno manual. <https://deno.land/manual>

European Parliament and Council. (2016). Regulation (EU) 2016/679, General Data Protection Regulation.

Framer. (2024). Motion documentation. <https://www.framer.com/motion>

Leaflet. (2024). Leaflet documentation. <https://leafletjs.com>

Ley 34/2002, de 11 de julio, de servicios de la sociedad de la información y de comercio electrónico.

Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales.

MDN Web Docs. (2024). Geolocation API. https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API

PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation. <https://www.postgresql.org/docs/16/>

React. (2024). React documentation. <https://react.dev>

React Router. (2024). React Router documentation. <https://reactrouter.com>

Recharts. (2024). Recharts documentation. <https://recharts.org>

Stripe. (2024). Stripe documentation. <https://stripe.com/docs>

Supabase. (2024). Supabase documentation. <https://supabase.com/docs>

Tailwind CSS. (2024). Tailwind CSS documentation. <https://tailwindcss.com/docs>

TypeScript. (2024). TypeScript documentation. <https://www.typescriptlang.org/docs>

Vercel. (2024). Vercel documentation. <https://vercel.com/docs>

Vite. (2024). Vite documentation. <https://vitejs.dev>

Wieruch, R. (2024). The Road to React. Leanpub. <https://www.roadtoreact.com>

Zustand. (2024). Zustand documentation. <https://zustand-demo.pmnd.rs>